

Form objects provide a method called `.as_p()` that outputs the form fields surrounded in paragraph (`<p>`) tags. Recall using `{{form.as_p}}` inside the login page, which is what rendered the fields in the form. To quickly test your new creation, open the REPL using the shell management command, create an instance of the `CommentForm`, and then print the results of `.as_p()`:

```
>>> from content.forms import CommentForm
>>> f = CommentForm()
>>> print(f.as_p())
<p>
  <label for="id_name">Name:</label>
  <input type="text" name="name" required id="id_name">
</p>
<p>
  <label for="id_comment">Comment:</label>
  <textarea name="comment" cols="50" rows="6" required
    id="id_comment"></textarea>
</p>
```

The results in the previous example were reformatted for readability. The actual content has more whitespace and newlines—Django forms are a bit messy with their indentation, but HTML doesn't care.

The "name" field in the `CommentForm` gets rendered as a `<label>` and `<input>` tag, while the "comment" field gets rendered as a `<label>` and `<textarea>`. Both fields have the `required` attribute, as this is the default. You can pass `required=False` to a field to turn this behavior off. Both fields also provide `id` and `name` attributes. This is important if you are planning on hand-tooling a form; you need to provide both of these attributes to process the form properly. More on this later. Finally, the `<textarea>` tag has the additional `rows` and `cols` attributes from the `attrs` dictionary provided.

With a form in place, you're all set to collect some data. You'll need a view for that.

7.1.1 Handling GET and POST in views

Up until now, your views have all been one-way: the user visits your URL, and your code provides some output. Recall that HTTP has multiple methods; the one you've been using is GET. When a form gets submitted by a web browser, the HTTP method is POST. The POST method almost never gets used on its own; typically, you perform a GET to fetch the page and its form, and then the browser submits the data back to you through a POST. The website validates the contents of the form, and if that validation fails, the same page gets displayed but with error information. If the validation passes, a different page gets displayed.

At first glance, this process sounds like you need multiple views. You can construct it that way, but the usual pattern is to use a single view and a redirect. With a single view handling both the GET and POST case, the rendered HTML doesn't need extra code in the `<form>` tag to tell it where to POST. It POSTs to the same place as the GET that rendered the page.